

Pre-commit

1

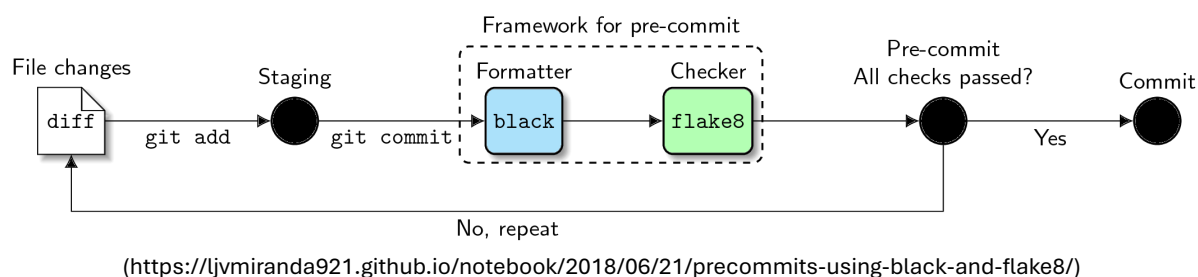
Git Commit Best Practices and Pre-Commit Hooks

- Commit often to manage smaller changes, making it easier to revert unwanted modifications.
- Mental tasks before committing include saving files, styling, formatting, and ensuring tests pass.
- These tasks may conflict with the principle of frequent commits.
- Pre-commit hooks can automate these tasks during each `git commit`.

2

Configuration

- Pre-commit simply works by inserting whatever workflow we want to automate in between whenever we do a `git commit` and afterwards would do a `git push`.
- The system works by looking for a file called `.pre-commit-config.yaml` that we can configure.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

3

3

A sample file

```
pre-commit sample-config | out-file .pre-commit-
config.yaml -encoding utf8
```

Result:

```
# See https://pre-commit.com for more information
# See https://pre-commit.com/hooks.html for more hooks
repos:
- repo: https://github.com/pre-commit/pre-commit-
  hooks
    rev: v3.2.0
    hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer
    - id: check-yaml
    - id: check-added-large-files
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

4

4

Pre-commit Configuration File and Execution

- Command: `pre-commit sample-config | out-file .pre-commit-config.yaml -encoding utf8`
- Result: A YAML file with pre-commit configuration is generated
- YAML file structure:
 - Lists repositories for pre-commits, e.g., `https://github.com/pre-commit/pre-commit-hooks`
 - Defines pre-commit hooks using `id`
- Example hooks in the file:
 - `trailing-whitespace`
 - `end-of-file-fixer`
 - `check-yaml`
 - `check-added-large-files`
- `id` corresponds to entries in `.pre-commit-hooks.yaml` found in the repository at the provided URL



5

Installing Pre-commit Configuration

- After defining `.pre-commit-config.yaml`, install it using:
`pre-commit install`
- This ensures the pre-commit hooks run automatically on each `git commit` execution



6

? Exercises

1. Install pre-commit
`pip install pre-commit`
2. Next create the sample file
`pre-commit sample-config > .pre-commit-config.yaml`
3. The sample file already contains 4 hooks. Make sure you understand what each do and if you need them at all.



7

4. `pre-commit` works by hooking into the `git commit` command, running whenever that command is run. For this to work, we need to install the hooks into `git commit`. Run
`pre-commit install`
to do this.
5. Try to commit your recently created `.pre-commit-config.yaml` file. You will likely not do anything, because `pre-commit` only check files that are being committed. Instead try to run
`pre-commit run --all-files`
that will check every file in your repository.



8

6. Try adding at least another check from the [base repository](#) to your `.pre-commit-config.yaml` file.
7. If you have learned about the linter `ruff`, `ruff` comes with its own [pre-commit hook](#). Try adding that to your `.pre-commit-config.yaml` file and see what happens when you try to commit files.
8. (Optional) Add more hooks to your `.pre-commit-config.yaml`.



9

9. Sometimes you are in a hurry, so make sure that you also can do commits without running `pre-commit` e.g.
`git commit -m <message> --no-verify`
10. Finally, figure out how to disable `pre-commit` again (if you get tired of it).



10